

COS341 2026: Continuation of Semester-Practical-Specification

Part 2a) *Semantic Analysis for Scopes and Names*

General Remark:

In this year's version of the student programming language **SPL** the rules for variable names and for function names are *reasonably similar to what you already know* from "the usual" languages such as "Java" or "C++". The finer details are specified below – all of them *illustrated by Fig.1* below.

VARIABLE NAMES

For any user-defined variable name, that occurs in some ALGO sub-tree (▲ in Fig.1) the following semantic rules shall be applicable:

- If x is some variable name under an ALGO at Scope Level l (with $l \geq 0$), then there must exist some V_DECL at a Scope Level $l' \leq l$ under the sub-tree of which the same name x is listed.
- If, for some variable name x under an ALGO at Scope Level l (with $l \geq 0$), there exist two different V_DECL at Scope Level l' and at Scope Level l'' which both contain declarations of x whereby $l'' < l' \leq l$, then the "nearer" declaration under the V_DECL at Scope Level l' is the "relevant" declaration for this x in this sub-tree of ALGO.
- At each Scope Level l (with $l \geq 0$), all user-defined-names in the sub-tree of V_DECL must differ from each other: no double-declarations at the same level.

This rule *also* applies in cases of **void/num funcName(V_DECL) { $P \dots$ }** where the local V_DECL in the sub-tree underneath P *may not "mask"* the V_DECL which are used as parameters between the round brackets behind the function's name.

Turn the page ►

FUNCTION NAMES

In *this year's version* of the **SPL**, no functional recursion shall be allowed – *neither* direct recursion (with *f* calling itself) *nor* indirect recursion (with *f* calling *g* and *g* back-calling *f*). This regulation motivates the following quite simple *semantic rules* for **strictly hierarchic** function calls:

- For a tree node *P* at Scope Level *l* (with $l \geq 0$), which has child-nodes **ALGO** and **F_DECL**, a function name *f* called in the sub-tree of **ALGO** must appear listed in the immediate sub-tree of **F_DECL** at the same Scope Level *l*.

[Comment: For the sake of simplicity we do not permit that two functions *f, g* both call a "shared" or "global" helper-function *h*. Otherwise we'd have "a hard time" **ensuring** that *h* will not anyhow recursively call back to *g* or *f*. Thus: if we wish to write an SPL program in which both *f* and *g* utilize the same helper-function *h*, then –unfortunately– we will have to program the function *h* in two copies, twice: once as a *strictly hierarchic* sub-function of *f*, and *again* as a *strictly hierarchic* sub-function *also* of *g*. From the perspective of the user this is "tedious" –having to write the same helper function twice– but from the perspective of the compiler engineer –i.e.: *You*– the implementation of the semantic analysis module is *much easier* this way. In this way we *also avoid other strange peculiarities* in which, for example, the main **ALGO** is *directly* calling some function *g* which can *only* be found as some *sub-sub-...sub-function* of function *f*.]

- For any given Scope Level *l*, all function names "underneath" **F_DECL** (at that level *l*) must be **unique**.

[Comment: for the sake of simplicity: no "**over-loading**" of function-names, even though *in theory* we could distinguish two different same-named functions by different parameters or by different return-types.]

Turn the page ►

UNIQUE RE-NAMING, SYMBOL TABLE, and ERROR MESSAGES

A **tree-crawling algorithm must be implemented** to such as detect whether a syntactically correct SPL program (which already has a syntax tree from the parser) is compliant with the above-given semantic rules. If any of the above-mentioned rules are violated, then your semantic analyser must "throw" an *appropriate error message* – *in particular if any variable name x or any function name f in any ALGO has no declaration, or if local variables in the "body" of a function are "masking" the same function's parameters.*

If, however, everything is semantically correct, then no error message may be "thrown". In such case, a *"static" Symbol Table must be "cleverly" utilized* to capture all the tree-nodes' Scope Level information which is needed to determine whether (or not) any two *tree-nodes with the same name n* are the *same computational* entities (from a semantic point of view), or whether they are different computational entities *in spite of* carrying the same name.

Unique internal renaming (with help of an auxiliary software) must then be applied, such that those tree-nodes, which are semantically the same, also carry the same new name, and those tree-nodes, which are semantically different, also carry different new system-generated names. For example: of four nodes with identical names n , two of them (being the same to each other) could perhaps get the new internal name **sys132** whilst the other two (being the same to each other but different from the foregoing two) could perhaps get the new internal name **sys421**. All these system-generated new names must –finally– also be stored in the Symbol Table (for later re-use by the code generator).

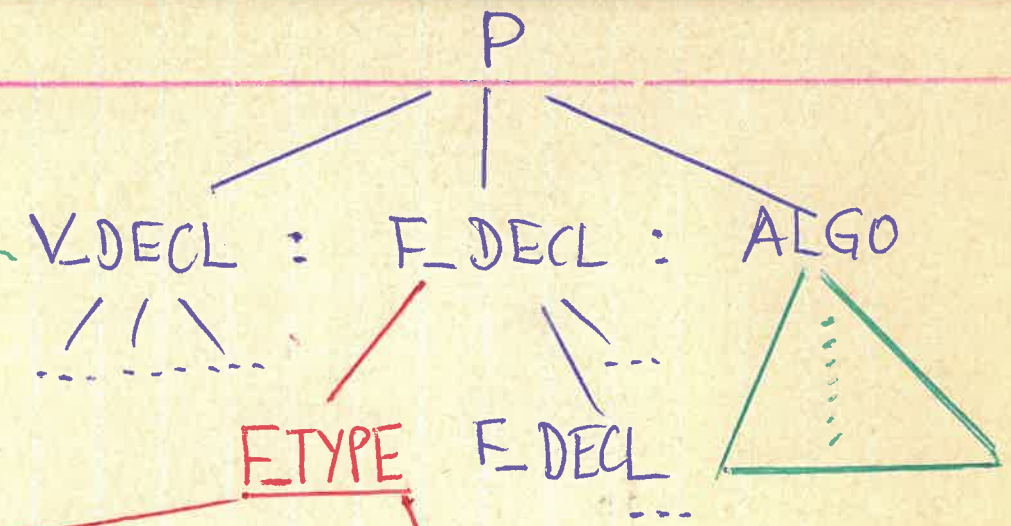
AND NOW...

Happy Coding :)

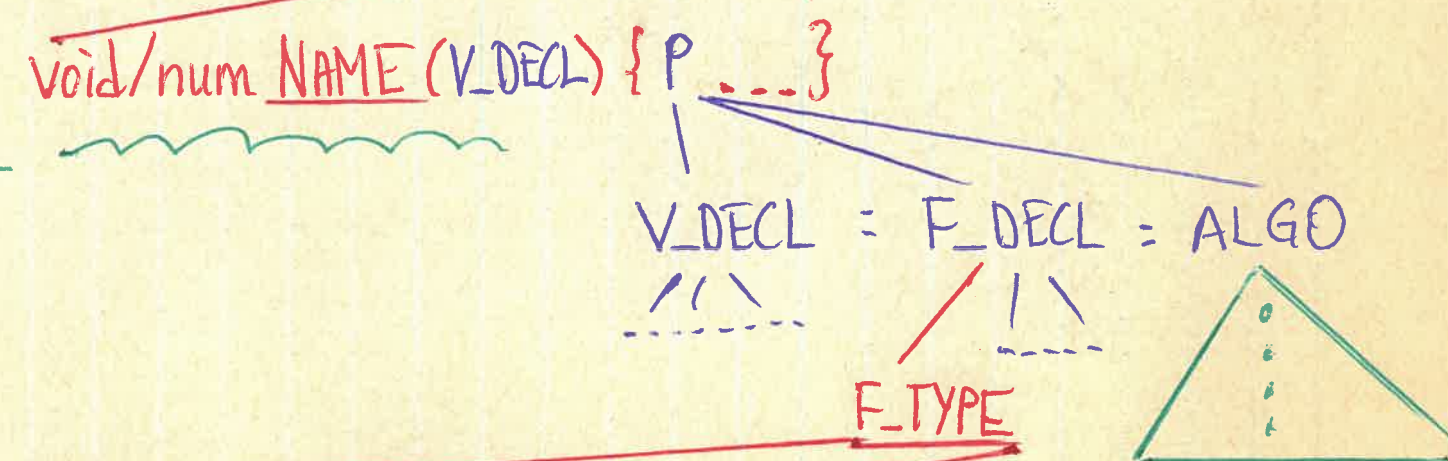
Turn the page for Fig.1 ►

Fig.1

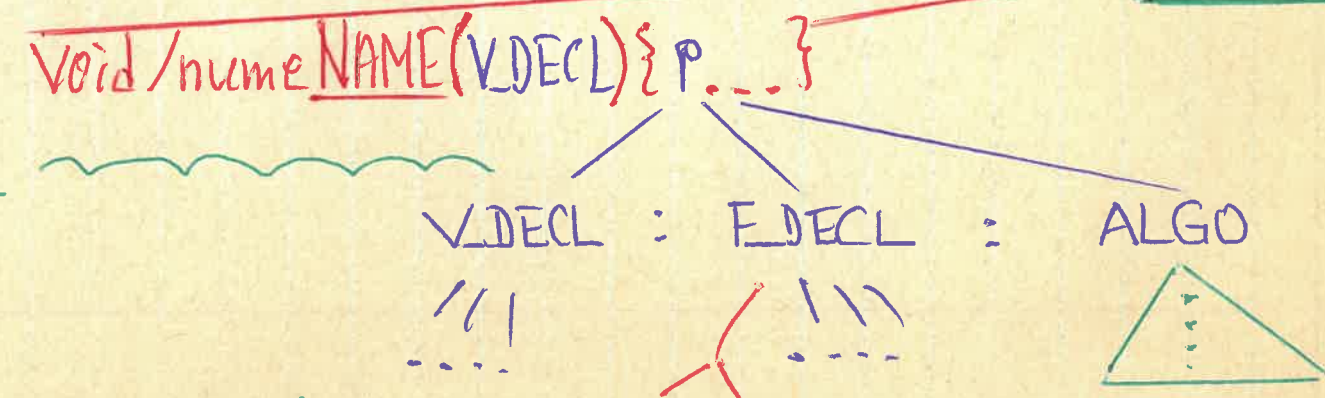
This is Scope Level 0



This is Scope Level 1



This is Scope Level 2



Etc... and so on... Scope level N...